# Assignment 3 - Integration Testing

## Work distribution

The work was divided between us both. Robin prepared the initial draft for the written explanation, and Mirnes reviewed and refined the text. The content was discussed and checked by us both.

## 1. Test Levels

### Difference in scope

Unit tests and integration tests differ mainly in scope.

A unit test focuses on one small part of the system in isolation, it could be something like a single method, function, or class. The purpose is to check whether that specific unit behaves correctly on its own.

An integration test focuses on how different parts of the system work together. Instead of testing one isolated unit, it tests the communication or cooperation between different components.

### Purpose of mocking in unit and integration tests

In unit testing, mocking is mainly used to isolate the unit under test.

This means that dependencies such as databases, APIs, or other classes are replaced by fake objects so that the test only checks the logic of the specific unit. Mocking also makes unit tests faster, more controlled, and easier to repeat, because the result does not depend on external systems.

For example, in Assignment 2 the DAO was mocked when testing `get_user_by_email()`. This allowed the test to focus only on the controller logic, without depending on whether the database was running or not.

But in integration testing, mocking has a different role.

The main goal of an integration test is to verify that multiple components actually work well together. Because of that, the integration should usually not be mocked away. If the communication between two components is replaced by mocks, then the real integration is no longer being tested.

Mocking can still be useful in integration tests, but only for parts that are outside the scope of the test or that would make the test difficult to control. The important part is that the main interaction must be real.

# 2. Integration Testing

For this part, we focused on the communication between `DAO.create()` and MongoDB. The goal was to check that object creation works correctly together with the validator of the user collection.

## Test design

We used the 4-step test design technique to derive the integration test cases. The ground truth for the test design is the assignment description, the documentation of `DAO.create()`, and the validator for the user collection.

### Step 1: Identify action and expected outcomes

| Action | Possible outcomes |
|---|---|
| Create a user using `DAO.create(user)` | the user is created successfully and returned with an `_id` / the creation fails with a database write error |

### Step 2: Identify conditions

The conditions that affect the result are:

| Condition | Possible values | Source |
|---|---|---|
| Required fields are present | yes / no | The user validator requires `firstName`, `lastName`, and `email`. |
| Data types are correct | yes / no | The user validator requires the user fields to have the correct BSON types, for example strings for `firstName`, `lastName`, and `email`. |
| Email already exists in the collection | yes / no | The `DAO.create()` documentation says values marked with `uniqueItems` should be unique, and the user validator marks `email` with `uniqueItems: true`. |

The optional `tasks` field is not used as a separate condition in these test cases because it is not required for creating a valid user. Omitting it is valid input and does not change the expected outcome for the core `DAO.create()` cases.

### Step 3: Determine combinations

With three conditions and two possible values each, the full combination space is 2 x 2 x 2 = 8 combinations. Some combinations can be collapsed because one failing validation condition is

already enough to make `DAO.create(user)` fail. In the table below, any means that the value does not affect the expected outcome for that row.

| # | Required fields present | Data types correct | Email already exists | `DAO.create(user)` |
|---|---|---|---|---|
| 1 | yes | yes | no | returns the created user with an `_id` |
| 2 | no | any | any | raises `pymongo.errors.WriteError` |
| 3 | yes | no | any | raises `pymongo.errors.WriteError` |
| 4 | yes | yes | yes | raises `pymongo.errors.WriteError` for the second insert |

### Step 4: Define expected outcomes

1. If all required fields are present, the data types are correct, and the email does not already exist, the user should be created and returned with an `_id`.
2. If a required field is missing, MongoDB should reject the user data and `DAO.create(user)` should raise `pymongo.errors.WriteError`.
3. If a field has the wrong data type, MongoDB should reject the user data and `DAO.create(user)` should raise `pymongo.errors.WriteError`.
4. If the email already exists, the second call to `DAO.create(user)` is expected to fail with `pymongo.errors.WriteError` according to the `DAO.create()` documentation.

## Final test cases

Based on the combinations above, the following integration tests were created:

1. Create a user with valid data. Expected outcome: the user is created and returned with an `_id`.
2. Create a user with a missing required field. Expected outcome: creation fails with a database write error.
3. Create a user with a wrong field type. Expected outcome: creation fails with a database write error.
4. Create two users with the same email. Expected outcome: the second creation should fail with a database write error.

## Pytest fixture

We implemented a pytest fixture that connects to a separate test collection called `user_test` in MongoDB. The fixture redirects the DAO to the test MongoDB URL, registers the user validator for

the test collection, creates a fresh collection before the test, and drops it after the test has finished. This allows the integration tests to interact with MongoDB without disturbing production code or data.

We do not mock the communication between `DAO.create()` and MongoDB, because that communication is the integration being tested. The fixture only controls the environment around the integration by using a test collection and test database configuration.

## Test implementation

The integration tests were implemented in:

https://github.com/mirNNes/bsv-edutask/blob/master/backend/test/integration/test_dao_create.py

## Test execution result

We ran the integration tests with pytest. The console output looked like this:

```
(.venv) MacBook-Air-som-tillhor-Robin:backend robin$ .venv/bin/python -m pytest -q
test/integration/test_dao_create.py
...F                                                                     [100%]
================================== FAILURES ===================================
_________________ test_create_user_fails_for_duplicated_email _________________

dao = <src.util.dao.DAO object at 0x102fcf500>

    @pytest.mark.integration
    def test_create_user_fails_for_duplicated_email(dao):
        first_user = {
            "firstName": "Jane",
            "lastName": "Doe",
            "email": "jane.doe@example.com",
        }
        second_user = {
            "firstName": "Janet",
            "lastName": "Doe",
            "email": "jane.doe@example.com",
        }

        dao.create(first_user)

>       with pytest.raises(pymongo.errors.WriteError):
E       Failed: DID NOT RAISE <class 'pymongo.errors.WriteError'>

test/integration/test_dao_create.py:86: Failed
---------------------------- Captured stdout setup -----------------------------
Connecting to collection user_test on MongoDB at url mongodb://localhost:27017
================================ tests coverage ================================
_________________ coverage: platform darwin, python 3.12.5-final-0 _________________

Name                                Stmts   Miss  Cover   Missing
-----------------------------------------------------------------
src/controllers/__init__.py             0      0   100%
src/controllers/controller.py          31     31     0%   1-103
src/controllers/taskcontroller.py      68     68     0%   1-139
src/controllers/todocontroller.py      21     21     0%   1-40
src/controllers/usercontroller.py      24     24     0%   1-46
src/util/dao.py                        67     34    49%   79-83, 101-118, 134-141, 156-162, 170-173
src/util/validators.py                  7      0   100%
-----------------------------------------------------------------
```

```
TOTAL                                 218    178    18%
========================== short test summary info ============================
FAILED test/integration/test_dao_create.py::test_create_user_fails_for_duplicated_email
1 failed, 3 passed in 0.36s
```

Three tests passed. This means that `DAO.create()` works as expected for valid input, missing required data, and wrong data types.

One test failed. Creating two users with the same email did not raise a write error. According to the `DAO.create()` documentation, values marked with `uniqueItems` should be unique among all documents in the collection, and the user validator marks `email` with `uniqueItems: true`. The failed test therefore shows that the current implementation or validator configuration does not enforce unique email addresses during user creation.